

DBMS Course Project Report

Student and College Management System

Campus Ledger

Team member	Roll number
Golamari Shashanth Reddy	25WU0101041
Hansini Baggu	25WU0101045
Kandibanda Balaji	25WU0101057

Course name and code: [ADD COURSE NAME AND CODE]

Faculty guide: [ADD FACULTY NAME]

Academic year: [ADD ACADEMIC YEAR]

Report prepared: 6 October 2026

2 Abstract

Campus Ledger is a local Student and College Management System that connects academic and fee records in a relational database. The problem addressed is repeated and inconsistent student data across admissions, course registration, attendance, examinations and accounts. The project uses a conceptual ER model, a MySQL 8 schema in Third Normal Form, integrity constraints and a role-based Flask web interface. Twenty-one base tables represent students, guardians, departments, programmes, courses, faculty, semesters, course offerings, registrations, class sessions, attendance, examinations, results, grades, fee bills and payments, with supporting tables for users and audit events. Four views derive reporting data.

The implemented interface supports insertion, deletion where records are unused, viewing, registrations, attendance, examination outcomes, grade publication, payment receipts and six CSV reports. A presentation database was populated with at least five synthetic records in each base table. The combined SQL file was executed on MySQL 8.0.46; a UI department record was inserted and deleted, with counts verified as 5, 6 and 5. The automated test suite completed with 45 passes, zero failures and zero errors. Demonstration attendance and grading values are sample rules, so institutional policy approval is required before real deployment.

3 Introduction and problem statement

A college must answer connected questions about one student: which programme the student joined, which course section they attend, whether an examination result is complete, and what fees remain unpaid. If each office maintains separate registers, information is repeated and reconciled manually. This increases the chance of inconsistent values and delays reporting.

The project therefore models college operations as related entities with one source of truth for each fact. A Student refers to a Programme; a Registration connects a Student to an actual Course Offering; Attendance and Exam Result rows attach to that registration; a Fee Bill belongs to a Student and its Payments determine the balance. The web interface gives each role a suitable view of these relationships.

4 Objectives and scope

- Record admissions and guardian relationships without repeating programme or department facts.
- Maintain courses, faculty, semesters, offerings and programme curriculum.
- Register students in eligible course sections while enforcing capacity and duplicate rules.
- Capture held sessions, attendance, examination results and final grades.
- Issue fee bills, record partial or full payments and derive outstanding balances.
- Provide role-specific access, audit records and six reports with CSV export.

Scope boundary. This academic prototype uses synthetic data and runs locally. Hostel, library, transport, payroll, gateway payments, refunds and automatic timetabling are not included. The 75% attendance threshold and grade scale are demonstration assumptions.

5 Software and hardware requirements

Component	Requirement or tested environment
Database	MySQL 8.0.46 tested; SQL uses MySQL 8 constraints and InnoDB transactions.
Application	Python 3.12 or later; Flask 3.1.3, PyMySQL 1.2.3, Waitress 3.0.2.
Testing	pytest 9.1.1 against a separate MySQL test database.
Client	Current desktop browser with HTML, CSS and JavaScript support.
Hardware	One local laptop or desktop able to run MySQL, Python and a browser. No performance minimum was benchmarked.
Connectivity	Localhost ports 3308 (MySQL) and 5080 (web app) for the demonstration.

6 ER diagram

The diagrams show the principal entities, primary keys, cardinalities and participation. In each one-to-many relationship, the child row requires its parent through a foreign key, while a parent can exist with zero child rows. The full-size PNG files are in the Presentation folder.

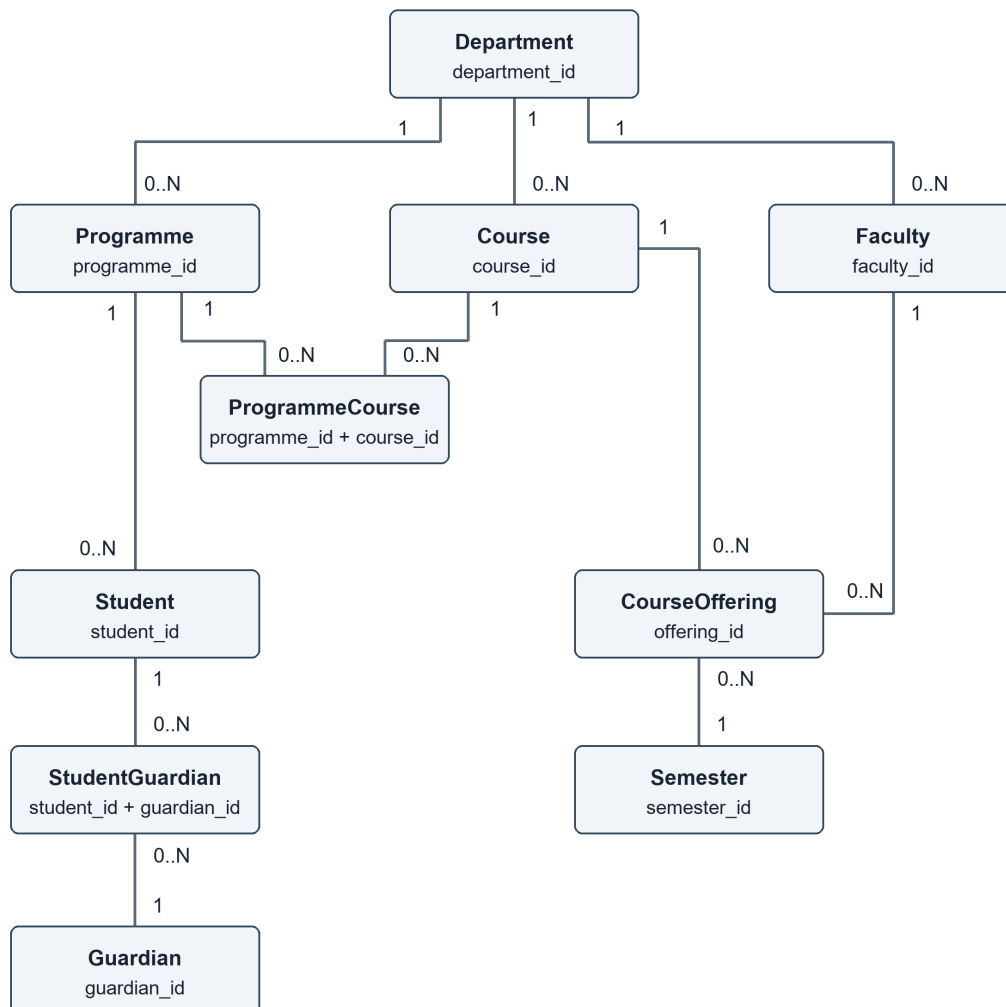


Figure 1. Academic structure and guardian link.

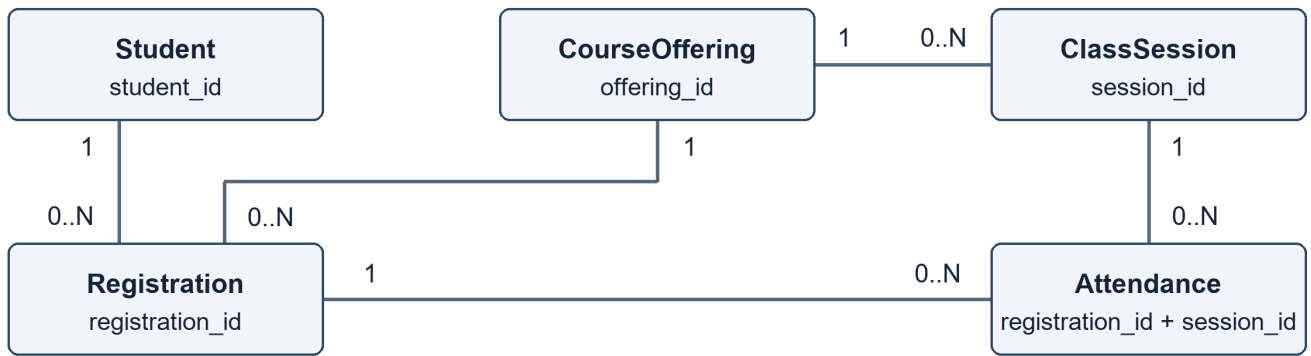


Figure 2. Registration, class sessions and attendance.



Figure 3. Examination results and final grades.

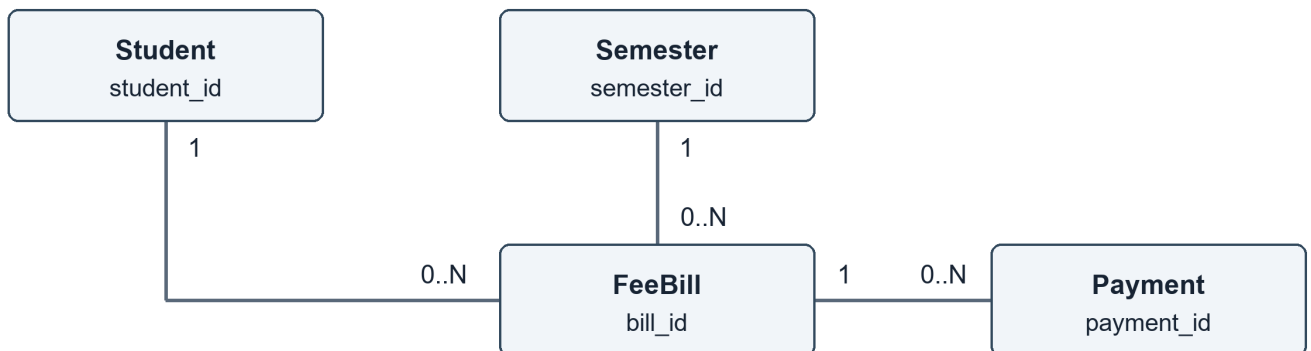


Figure 4. Fee bills and payments.

7 Relational schema and normalization

The schema has 21 base tables. Primary keys identify rows, foreign keys connect dependent rows, unique keys protect business identifiers, and checks reject invalid ranges or states. Representative relations are Student(student_id PK, programme_id FK, registration_no UNIQUE), CourseOffering(offering_id PK, course_id FK, semester_id FK, faculty_id FK), Registration(registration_id PK, student_id FK, offering_id FK), Attendance(registration_id + session_id composite PK), FeeBill(bill_id PK, student_id FK, semester_id FK), and Payment(payment_id PK, bill_id FK).

Table	Primary key	Parent table(s)	Purpose
app_user	user_id	department / faculty / student	Role-linked application sign-in
attendance	registration_id, session_id	class_session / registration	Presence for a held class
audit_event	event_id	app_user	Successful action history
class_session	session_id	course_offering	One held class
course	course_id	department	Reusable course catalogue
course_offering	offering_id	course / faculty / semester	Section in a semester
department	department_id	—	Academic department
exam_result	registration_id, exam_id	examination / registration	Student assessment outcome
examination	exam_id	course_offering	Assessment definition
faculty	faculty_id	department	Faculty master
fee_bill	bill_id	semester / student	Student fee demand
grade	registration_id	grade_scale / registration	Final course grade
grade_scale	letter_grade	—	Demonstration grade thresholds
guardian	guardian_id	—	Guardian contact
payment	payment_id	fee_bill	Confirmed fee receipt
programme	programme_id	department	Programme within department
programme_course	programme_id, course_id	course / programme	Programme curriculum
registration	registration_id	course_offering / student	Student course enrollment
semester	semester_id	—	Academic period
student	student_id	programme	Admission and identity
student_guardian	student_id, guardian_id	guardian / student	Student and guardian link

Third Normal Form justification

Each table stores one subject and every non-key attribute describes its key. Student stores programme_id, while the department comes through Programme, avoiding a transitive dependency. Many-to-many associations are represented by StudentGuardian, ProgrammeCourse and Registration. Repeated events such as class sessions, results and payments are separate rows. Balances and attendance percentages are calculated in views rather than stored as editable duplicates.

8 Data dictionary

This dictionary is generated from the executed MySQL schema. PK identifies a primary key; FK names a referenced table; UNIQUE identifies an alternate key. The full DDL, including CHECK constraints and indexes, is in the combined SQL file.

app_user

Column	Type	Null?	Key / extra
user_id	int	NO	PK, auto_increment
username	varchar(60)	NO	UNIQUE
password_hash	varchar(255)	NO	—
role	varchar(12)	NO	—
student_id	int	YES	UNIQUE, FK → student
faculty_id	int	YES	UNIQUE, FK → faculty
department_id	int	YES	FK → department

attendance

Column	Type	Null?	Key / extra
registration_id	int	NO	PK, FK → registration
session_id	int	NO	PK, FK → class_session
attendance_status	varchar(8)	NO	—

audit_event

Column	Type	Null?	Key / extra
event_id	int	NO	PK, auto_increment
user_id	int	YES	FK → app_user
action	varchar(60)	NO	—
detail	varchar(255)	NO	—
created_at	timestamp	NO	DEFAULT_GENERATED

class_session

Column	Type	Null?	Key / extra
session_id	int	NO	PK, auto_increment
offering_id	int	NO	FK → course_offering
session_date	date	NO	—
session_no	int	NO	—
topic	varchar(200)	YES	—

course

Column	Type	Null?	Key / extra
course_id	int	NO	PK, auto_increment
department_id	int	NO	FK → department
course_code	varchar(20)	NO	UNIQUE
course_title	varchar(150)	NO	—
credits	decimal(3,1)	NO	—

course_offering

Column	Type	Null?	Key / extra
offering_id	int	NO	PK, auto_increment
course_id	int	NO	FK → course
semester_id	int	NO	FK → semester
faculty_id	int	NO	FK → faculty
section_code	varchar(12)	NO	—
capacity	int	NO	—

department

Column	Type	Null?	Key / extra
department_id	int	NO	PK, auto_increment
department_code	varchar(12)	NO	UNIQUE
department_name	varchar(120)	NO	—

exam_result

Column	Type	Null?	Key / extra
registration_id	int	NO	PK, FK → registration
exam_id	int	NO	PK, FK → examination
result_status	varchar(8)	NO	—
marks_obtained	decimal(6,2)	YES	—

examination

Column	Type	Null?	Key / extra
exam_id	int	NO	PK, auto_increment
offering_id	int	NO	FK → course_offering
exam_name	varchar(60)	NO	—
exam_date	date	NO	—
max_marks	decimal(6,2)	NO	—
weight_percent	decimal(5,2)	NO	—

faculty

Column	Type	Null?	Key / extra
faculty_id	int	NO	PK, auto_increment
department_id	int	NO	FK → department
employee_no	varchar(20)	NO	UNIQUE
full_name	varchar(100)	NO	—
email	varchar(120)	YES	—
phone	varchar(20)	YES	—

fee_bill

Column	Type	Null?	Key / extra
bill_id	int	NO	PK, auto_increment
student_id	int	NO	FK → student
semester_id	int	NO	FK → semester
bill_no	varchar(32)	NO	UNIQUE
description	varchar(150)	NO	—
issued_on	date	NO	—
due_date	date	NO	—
total_amount	decimal(12,2)	NO	—

grade

Column	Type	Null?	Key / extra
registration_id	int	NO	PK, FK → registration
letter_grade	varchar(3)	NO	FK → grade_scale
finalized_on	date	NO	—

grade_scale

Column	Type	Null?	Key / extra
letter_grade	varchar(3)	NO	PK
minimum_score	decimal(5,2)	NO	UNIQUE
grade_points	decimal(3,1)	NO	—

guardian

Column	Type	Null?	Key / extra
guardian_id	int	NO	PK, auto_increment
full_name	varchar(100)	NO	—
phone	varchar(20)	NO	—
email	varchar(120)	YES	—

payment

Column	Type	Null?	Key / extra
payment_id	int	NO	PK, auto_increment
bill_id	int	NO	FK → fee_bill
receipt_no	varchar(40)	NO	UNIQUE
paid_on	date	NO	—
amount	decimal(12,2)	NO	—
payment_mode	varchar(16)	NO	—
transaction_reference	varchar(100)	YES	—

programme

Column	Type	Null?	Key / extra
programme_id	int	NO	PK, auto_increment
department_id	int	NO	FK → department
programme_code	varchar(20)	NO	UNIQUE
programme_name	varchar(120)	NO	—
duration_terms	int	NO	—

programme_course

Column	Type	Null?	Key / extra
programme_id	int	NO	PK, FK → programme
course_id	int	NO	PK, FK → course
recommended_term	int	NO	—
course_type	varchar(10)	NO	—

registration

Column	Type	Null?	Key / extra
registration_id	int	NO	PK, auto_increment
student_id	int	NO	FK → student
offering_id	int	NO	FK → course_offering
registered_on	date	NO	—
registration_status	varchar(10)	NO	—
dropped_on	date	YES	—

semester

Column	Type	Null?	Key / extra
semester_id	int	NO	PK, auto_increment
academic_year	varchar(12)	NO	—
term_name	varchar(20)	NO	—
start_date	date	NO	—
end_date	date	NO	—

student

Column	Type	Null?	Key / extra
student_id	int	NO	PK, auto_increment
programme_id	int	NO	FK → programme
registration_no	varchar(24)	NO	UNIQUE
full_name	varchar(100)	NO	—
date_of_birth	date	NO	—
email	varchar(120)	YES	—
phone	varchar(20)	YES	—
admission_date	date	NO	—
student_status	varchar(12)	NO	—

student_guardian

Column	Type	Null?	Key / extra
student_id	int	NO	PK, FK → student
guardian_id	int	NO	PK, FK → guardian
relationship_type	varchar(30)	NO	—

9 SQL commands used with sample outputs

The file **Presentation/DBMS_Course_Project_All_Commands.sql** is runnable in a fresh MySQL 8 database. It includes database creation, 21 table definitions, procedures, triggers, views, synthetic INSERT statements and SELECT queries. The MySQL execution transcript is supplied as **SQL_Execution_Output.txt**.

```
CREATE DATABASE college_pbl_presentation
  CHARACTER SET utf8mb4;
CREATE TABLE department (
  department_id INT AUTO_INCREMENT PRIMARY KEY,
  department_code VARCHAR(12) NOT NULL UNIQUE,
  department_name VARCHAR(120) NOT NULL
);
INSERT INTO department (department_code, department_name)
VALUES ('CSE', 'Computer Science and Engineering');
```

The presentation script then adds synthetic rows to satisfy the five-per-table requirement. A final SELECT verified these counts after import:

Base table	Rows	Base table	Rows
app_user	7	grade	9
attendance	83	grade_scale	7
audit_event	7	guardian	11
class_session	12	payment	9
course	5	programme	5
course_offering	5	programme_course	5
department	5	registration	21
exam_result	41	semester	5
examination	6	student	12
faculty	5	student_guardian	12
fee_bill	12		

The temporary UI demonstration row was removed after capture; the department count therefore remains five. Audit events may increase when the UI is used.

10 Queries with outputs

Join across student, registration, offering and course

```
SELECT s.registration_no, s.full_name, c.course_code, o.section_code
FROM student s JOIN registration r ON r.student_id=s.student_id
JOIN course_offering o ON o.offering_id=r.offering_id
JOIN course c ON c.course_id=o.course_id
ORDER BY s.registration_no, c.course_code LIMIT 6;
```

Registration	Student	Course	Section
2026CS001	Aarav Sharma	CS301	A
2026CS001	Aarav Sharma	CS302	A
2026CS002	Diya Patel	CS301	A
2026CS002	Diya Patel	CS302	A
2026CS003	Ishaan Verma	CS301	A
2026CS003	Ishaan Verma	CS302	A

Aggregate query including empty course sections

```
SELECT c.course_code, o.section_code, COUNT(r.registration_id) AS active_students
FROM course_offering o JOIN course c ON c.course_id=o.course_id
LEFT JOIN registration r ON r.offering_id=o.offering_id
    AND r.registration_status='ACTIVE'
GROUP BY o.offering_id,c.course_code,o.section_code;
```

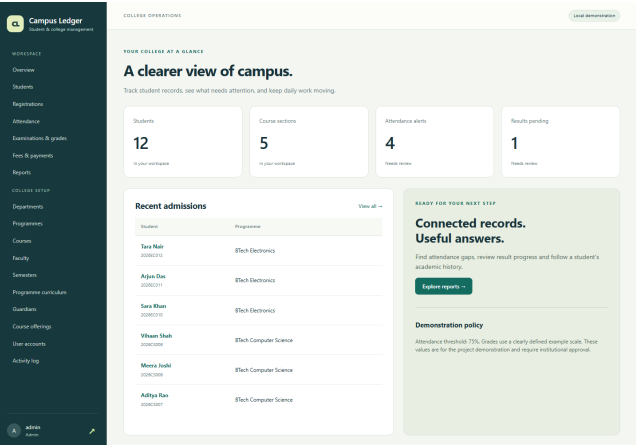
Course	Section	Active students
CS301	A	9
CS301	B	0
CS302	A	9
CS303	A	0
EC301	A	3

Derived fee balances from a reporting view

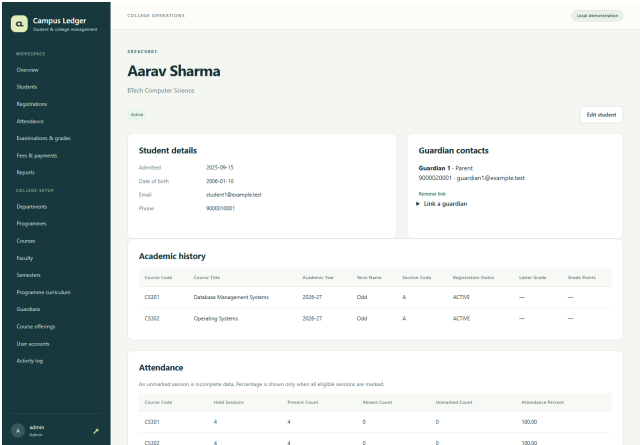
Bill	Student	Total	Paid	Balance
BILL-001	Aarav Sharma	20,000.00	20,000.00	0.00
BILL-002	Diya Patel	20,000.00	8,000.00	12,000.00
BILL-003	Ishaan Verma	20,000.00	20,000.00	0.00
BILL-004	Ananya Singh	20,000.00	8,000.00	12,000.00
BILL-005	Rohan Gupta	20,000.00	8,000.00	12,000.00

11 UI design and screenshots

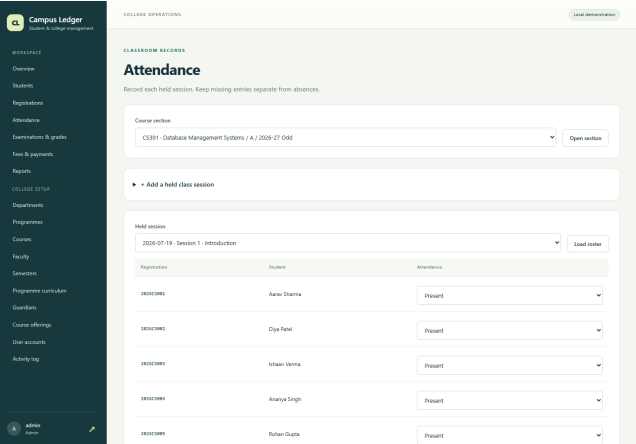
The interface exposes role-specific pages for records, teaching, examinations, accounts and reports. The following screenshots were captured from the running application connected to the presentation MySQL database. A separate set in the Presentation/screenshots folder covers all main UI screens.



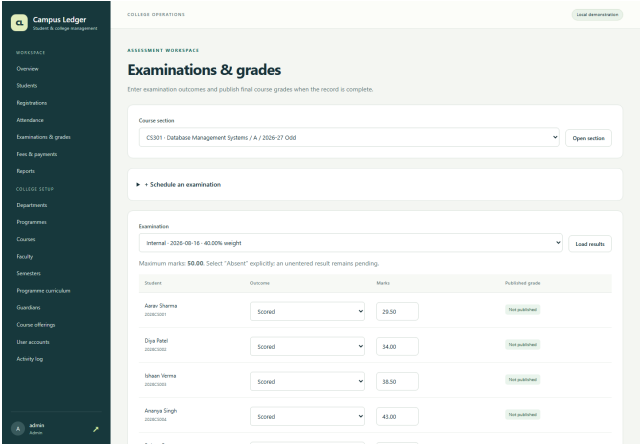
Dashboard: live summary.



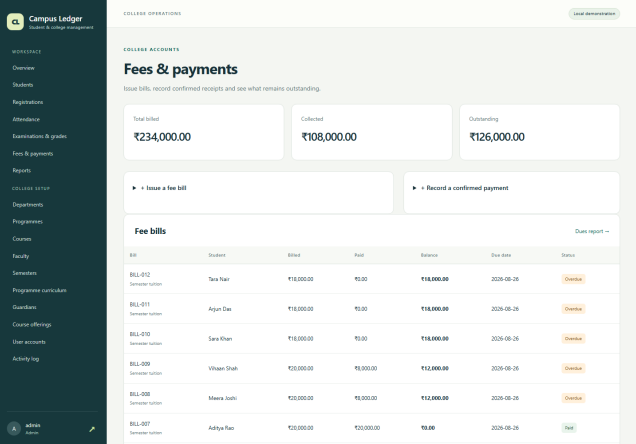
Student profile: connected records.



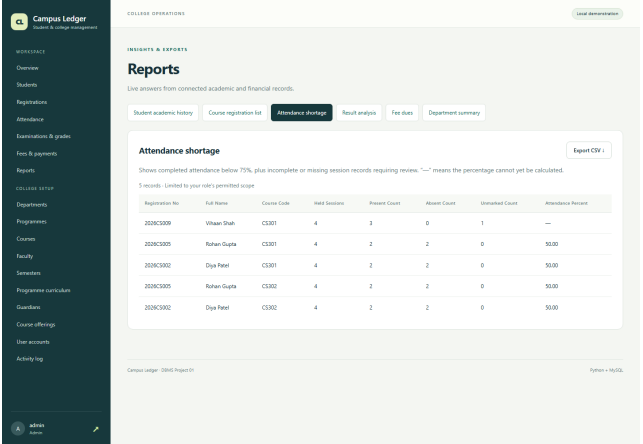
Attendance entry.



Examination outcome entry.

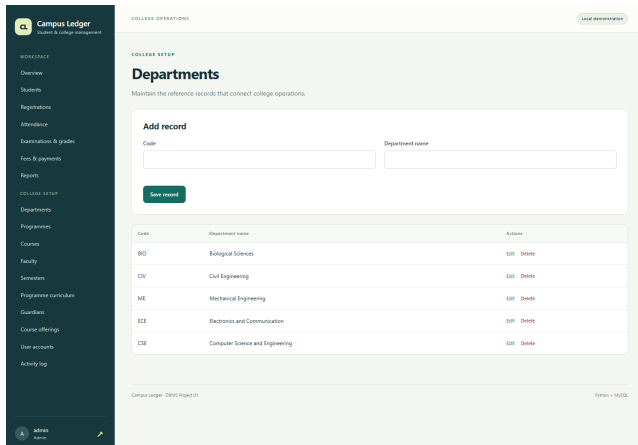


Fee bills and payments.

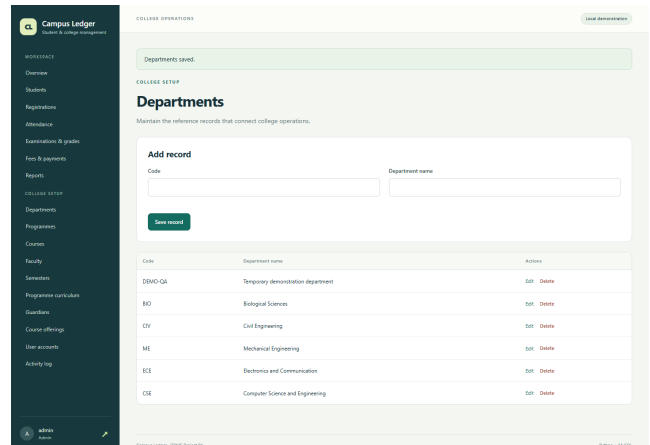


Attendance and other reports.

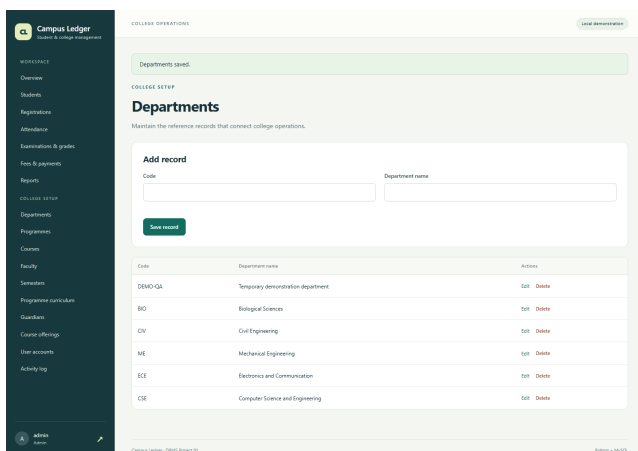
Before and after insertion and deletion



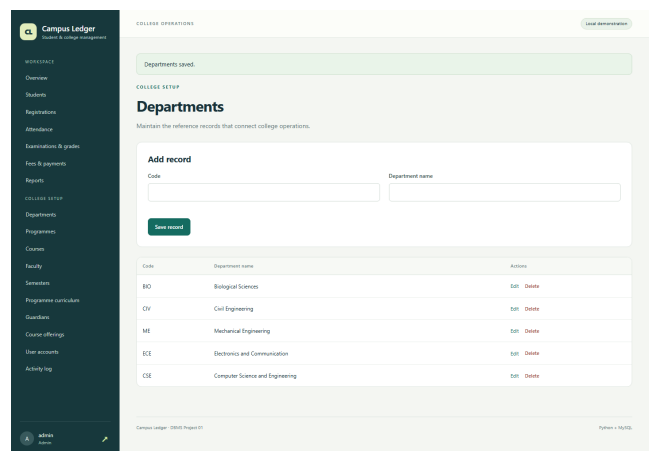
Before insert: five departments.



After insert: DEMO-QA shown, six departments.



Before delete: DEMO-QA is present.



After delete: DEMO-QA is absent.

The UI create and delete actions changed the live department count from 5 to 6 and back to 5; the exact SELECT evidence is in **Presentation/CRUD_Database_Evidence.txt**.

12 Implementation details

Layer	Implementation
Browser	HTML templates, CSS, JavaScript forms and CSV downloads.
Application	Flask routes, role checks, CSRF tokens, validation and audit writes.
Database access	PyMySQL parameterized queries and transaction helper.
Database	MySQL 8 with InnoDB tables, foreign keys, checks, views, procedures and triggers.

The application reads a local configuration file that is excluded from Git. The database helper opens a PyMySQL connection using that configuration; SQL values are passed as parameters. Mutating operations use transactions. Registration and payment stored procedures lock the relevant rows with SELECT FOR UPDATE so concurrent requests cannot exceed section capacity or bill balance.

```
def query(sql, args=(), one=False, con=None):
    own = con is None
    con = con or connect()
    try:
        with con.cursor() as cur:
            cur.execute(sql, args)
            return cur.fetchone() if one else cur.fetchall()
    finally:
        if own: con.close()
```

The UI module source is in **Presentation/UI-Source**; the complete SQL is in **Presentation/DBMS_Course_Project_All_Commands.sql**.

13 Testing: cases and results

The automated suite was run against a separate MySQL test database on 6 October 2026: **45 tests passed, 0 failures, 0 errors**. The result file is Presentation/UI-Source/docs/test-results-final.xml. The main presentation database was not reset by the tests.

Test area	Expected result	Observed
Role permissions	Unauthorized account cannot access another role's page	Pass
CSRF and input validation	Invalid or missing token/value is rejected	Pass
Course registration	Eligibility, duplicate and capacity rules enforced	Pass
Concurrent last seat	Only one competing registration succeeds	Pass
Attendance and marks	Mismatched session or excess marks rejected	Pass
Grade publication	Incomplete results prevent publication	Pass
Concurrent payment	Overpayment cannot be recorded	Pass
Reports and CSV	Expected rows and export delivered	Pass
UI CRUD demonstration	Department count changes 5 → 6 → 5	Pass

14 Conclusion and future enhancements

The completed prototype turns separate college records into linked, queryable data. MySQL constraints protect relationships; views compute report values; the Flask interface supports the main academic and fee workflows. The fresh presentation database met the five-row minimum for every table, the UI insert/delete actions were reflected in MySQL, and the automated suite passed.

- Replace demonstration attendance and grading rules with approved institutional policy.

- Add managed deployment, HTTPS, backup monitoring and a tested recovery process.
- Add institution-verified identity, password recovery and account lifecycle controls.
- Extend the scope to modules such as library, hostel or timetable only after requirements review.

15 References

- [1] DBMS Course Project, Final Presentation Instructions and Schedule, supplied PDF, 4 October 2026.
- [2] DBMS PBL 70 Unique Project Statements, supplied Word document, Project 01.
- [3] Campus Ledger project source: sql/01_schema.sql, sql/02_routines.sql, sql/03_views_reports.sql, app.py, db.py and tests/test_system.py.
- [4] MySQL 8.0 Reference Manual, CREATE TABLE Statement, <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>
- [5] Flask Documentation 3.1, <https://flask.palletsprojects.com/en/stable/>
- [6] PyMySQL Documentation, <https://pymysql.readthedocs.io/en/latest/>

16 Contribution of each member

Complete this section with actual work and matching GitHub commits. The team requested editable placeholders; no contribution is assigned without confirmation.

Member	Actual contribution	Commit evidence
Golamari Shashanth Reddy (25WU0101041)	[ADD ACTUAL CONTRIBUTION]	[ADD COMMIT LINK]
Hansini Baggu (25WU0101045)	[ADD ACTUAL CONTRIBUTION]	[ADD COMMIT LINK]
Kandibanda Balaji (25WU0101057)	[ADD ACTUAL CONTRIBUTION]	[ADD COMMIT LINK]

17 Appendix: GitHub repository link

Public repository: <https://github.com/gshashanthr938-ai/DBMS-Course-Project>

Repository owner: gshashanthr938-ai

Other members as collaborators: hansini215-crypto, balajikandibanda123-eng

The public repository contains the Presentation and Project-Report folders. Add the other two members as collaborators and include their actual commits before final submission.